Power Car Seat - Panthera Project

ABSTRACT

This document provides comprehensive technical documentation for the Panthera firmware, detailing its architecture, hardware-software interface, protection mechanisms, state machines, and API summary. It serves as a reference for developers and engineers involved in the design, implementation, and maintenance of the firmware, ensuring clarity and consistency throughout the development lifecycle.

Table of Contents

- **1. Overview**
- **2. Hardware-software Interface**
  - **2.1 Power Pins**
  - **2.2 Gpio Pins**
  - **2.3 Adc Pins**
  - **Pin Mapping Table**
- **3. System Architecture**
  - **3.1 System Diagram**
- **4. Protection**
- **5. Detail State Machine**
  - **5.1 IDLE State**
  - **5.3 AUTO State**
  - **5.4 SAVE State**
  - **5.5 HOMING State**
  - **5.6 FAULT State**
  - **5.7 AUTORECOVERY State**
  - **5.8 SHUTDOWN State**
- **6. Api Summary**
  - **6.1 Driver**
  - **6.2 Middleware**
  - **6.3 Application**
- **Revision Note**

1. Overview

Describe the general purpose, scope, and goals of the firmware.

2. Hardware-software Interface

2.1 Power Pins

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
11	VDD_LV_FLA/VDD_LV	POWER	In	VDD_LV_FLA/VDD_LV	Power Supply	5V
12	VDD_HV_IO_MAIN_1	POWER	In	VDD_HV_IO_MAIN_1	Power Supply	5V
19	VSS_HV_ADR_S	POWER	In	VSS_HV_ADR_S	Ground of power supply	0V
20	VDD_HV_ADR_S	POWER	In	VDD_HV_ADR_S	Power Supply	5V
21	VSS_HV_ADV_S	POWER	In	VSS_HV_ADV_S	Ground of power supply	0V
22	VDD_HV_ADV_S	POWER	In	VDD_HV_ADV_S	Power Supply	5V
27	VDD_LV_ADC_S/VDD_LV	POWER	In	VDD_LV_ADC_S/VDD_LV	Power Supply	5V
28	VDD_HV_IO_MAIN_2	POWER	In	VDD_HV_IO_MAIN_2	Power Supply	5V
33	VDD_LV_1	POWER	In	VDD_LV_1	Power Supply	5V
34	VDD_HV_IO_MAIN_3	POWER	In	VDD_HV_IO_MAIN_3	Power Supply	5V
35	VSS_HV_OSC	POWER	In	VSS_HV_OSC	Ground of power supply	0V
38	VDD_HV_OSC	POWER	In	VDD_HV_OSC	Power Supply	5V
47	VDD_LV_2	POWER	In	VDD_LV_2	Power Supply	5V
49	VDD_HV_IO_MAIN_4	POWER	In	VDD_HV_IO_MAIN_4	Power Supply	5V
57	VDD_HV_FLA/VDD_HV_IO_MAIN	POWER	In	VDD_HV_FLA/VDD_HV_IO_MAIN	Power Supply	5V
65	EP	POWER	In/Out	EP	GND	0V

2.2 Gpio Pins

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
1	PD[14]	GPIO	In	RECLIER_1	Return seatback upright	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
2	PD[15]	GPIO	In	RECLIER_2	Seatback reclining	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
3	PC[4]	GPIO	Out	INB_3	Signal used to detect the rotation direction of the motor 2	Logic 0 (0V) Logic 1 (5V)
4	PC[3]	GPIO	Out	INA_3	Signal used to detect the rotation direction of the motor 3	Logic 0 (0V) Logic 1 (5V)
5	PC[2]	GPIO	Out	SEL0_3	Configure MultiSense Monioring for Motor 3	Logic 0 (0V) Logic 1 (5V)
6	PC[1]	GPIO	Out	MulEN_3	Enable/ Disable MultiSense Monitoring Motors 3	Logic 0 (0V) Logic 1 (5V)
8	PD[12]	GPIO	Out	PWM_1	PWM of Motor 1	Logic 0 (0V) Logic 1 (5V)
9	PD[13]	GPIO	In	SLIDER_1	Seat Forward Adjustment Button	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
10	PE[3]	GPIO	In	SLIDER_2	Seat Backward Adjustment Button	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
13	PI[1]	GPIO	In	HIEGHT_1	Raise the seat	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
14	PI[2]	GPIO	In	HIEGHT_2	Lower the seat	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
15	PI[3]	GPIO	In	RECALL_3	Auto move seat to position 3	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
16	PI[4]	GPIO	In	RECALL_2	Auto move seat to position 2	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
17	PI[6]	GPIO	In	SAVE	Save position	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
18	PI[7]	GPIO	In	RECALL_1	Auto move seat to position 1	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
23	PG[10]	GPIO	In	HALL_1	Hall Signal Motor 1	Logic 0 (0V): Low Logic 1 (5V): High
24	PG[11]	GPIO	In	HALL_2	Hall Signal Motor 2	Logic 0 (0V): Low Logic 1 (5V): High
29	PB[11]	GPIO	Out	INB_2	Signal used to detect the rotation direction of the motor 2	Logic 0 (0V) Logic 1 (5V)
31	PB[9]	GPIO	Out	INA_2	Signal used to detect the rotation direction of the motor 2	Logic 0 (0V) Logic 1 (5V)
46	PM[14]	GPIO	Out	INA_1	Signal used to detect the rotation direction of the motor 1	Logic 0 (0V) Logic 1 (5V)
48	PD[5]	GPIO	Out	INB_1	Signal used to detect the rotation direction of the motor 1	Logic 0 (0V) Logic 1 (5V)
52	PA[10]	GPIO	Out	MulEN_2	Enable/ Disable MultiSense Monitoring Motor 2	Logic 0 (0V) Logic 1 (5V)
53	PA[11]	GPIO	Out	SEL0_2	Configure MultiSense Monioring for Motor 2	Logic 0 (0V) Logic 1 (5V)
54	PE[10]	GPIO	Out	PWM_3	PWM of Motor 2	Logic 0 (0V) Logic 1 (5V)
55	PE[11]	GPIO	Out	SEL1_3	Configure MultiSense	Logic 0 (0V) Logic 1 (5V)

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
					Monioring for Motor 3	
56	PH[4]	GPIO	Out	SEL0_1	Configure MultiSense Monioring for Motor 1	Logic 0 (0V) Logic 1 (5V)
58	PD[0]	GPIO	In	TW	Advanced Thermal warning output	Logic 0 (0V): Low Logic 1 (5V): High
59	PC[15]	GPIO	Out	SEL1_2	Configure MultiSense Monioring for Motor 2	Logic 0 (0V) Logic 1 (5V)
60	PC[14]	GPIO	Out	SEL1_1	Configure MultiSense Monioring for Motor 1	Logic 0 (0V) Logic 1 (5V)
61	PC[13]	GPIO	Out	PWM_2	PWM of Motor 2	Logic 0 (0V) Logic 1 (5V)
62	PC[12]	GPIO	In	HALL_3	Hall Signal Motor 3	Logic 0 (0V): Low Logic 1 (5V): High
63	PF[3]	GPIO	Out	MulEN_1	Enable/ Disable MultiSense Monitoring Motor 1	Logic 0 (0V) Logic 1 (5V)

2.3 Adc Pins

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
25	PG[12]	ADC	In	5V_FB	Feedback LDO 5V to check Power Supply	0V - 5V
26	PD[11]	ADC	In	MulSens_3	Current input of Motor 3	0V/0A - 5V/22.5A, Max Software: 20V
30	PB[10]	ADC	In	MulSens_2	Current input of Motor 2	0V/0A - 5V/22.5A, Max Software: 20V
32	PB[8]	ADC	In	MulSens_1	Current input of Motor 1	0V/0A - 5V/25A, Max Software: 20V

Pin Mapping Table

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
1	PD[14]	GPIO	In	RECLIER_1	Return seatback upright	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
2	PD[15]	GPIO	In	RECLIER_2	Seatback reclining	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
3	PC[4]	GPIO	Out	INB_3	Signal used to detect the rotation direction of the motor 2	Logic 0 (0V) Logic 1 (5V)
4	PC[3]	GPIO	Out	INA_3	Signal used to detect the rotation direction of the motor 3	Logic 0 (0V) Logic 1 (5V)
5	PC[2]	GPIO	Out	SEL0_3	Configure MultiSense Monioring for Motor 3	Logic 0 (0V) Logic 1 (5V)
6	PC[1]	GPIO	Out	MulEN_3	Enable/ Disable MultiSense Monitoring Motors 3	Logic 0 (0V) Logic 1 (5V)
7	PE[2]	Reset	In	RST	Reset MCU	None
8	PD[12]	GPIO	Out	PWM_1	PWM of Motor 1	Logic 0 (0V) Logic 1 (5V)
9	PD[13]	GPIO	In	SLIDER_1	Seat Forward Adjustment Button	Logic 0 (0V): Button Released

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
						Logic 1 (5V): Button Pressed
10	PE[3]	GPIO	In	SLIDER_2	Seat Backward Adjustment Button	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
11	VDD_LV_FLA/VDD_LV	POWER	In	VDD_LV_FLA/VDD_LV	Power Supply	5V
12	VDD_HV_IO_MAIN_1	POWER	In	VDD_HV_IO_MAIN_1	Power Supply	5V
13	PI[1]	GPIO	In	HIEGHT_1	Raise the seat	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
14	PI[2]	GPIO	In	HIEGHT_2	Lower the seat	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
15	PI[3]	GPIO	In	RECALL_3	Auto move seat to position 3	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
16	PI[4]	GPIO	In	RECALL_2	Auto move seat to position 2	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
17	PI[6]	GPIO	In	SAVE	Save position	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
18	PI[7]	GPIO	In	RECALL_1	Auto move seat to position 1	Logic 0 (0V): Button Released Logic 1 (5V): Button Pressed
19	VSS_HV_ADR_S	POWER	In	VSS_HV_ADR_S	Ground of power supply	0V
20	VDD_HV_ADR_S	POWER	In	VDD_HV_ADR_S	Power Supply	5V
21	VSS_HV_ADV_S	POWER	In	VSS_HV_ADV_S	Ground of power supply	0V
22	VDD_HV_ADV_S	POWER	In	VDD_HV_ADV_S	Power Supply	5V

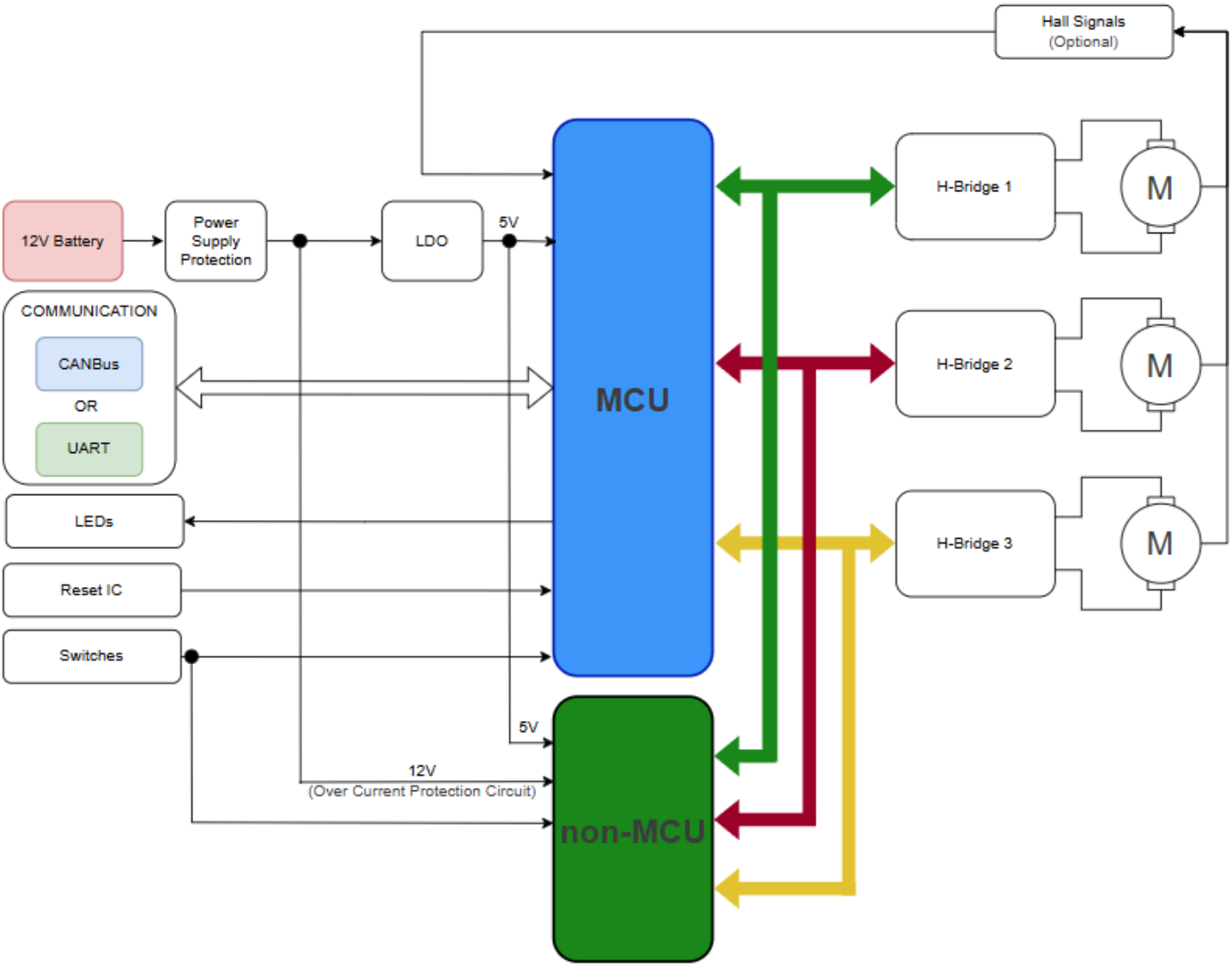
Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
23	PG[10]	GPIO	In	HALL_1	Hall Signal Motor 1	Logic 0 (0V): Low Logic 1 (5V): High
24	PG[11]	GPIO	In	HALL_2	Hall Signal Motor 2	Logic 0 (0V): Low Logic 1 (5V): High
25	PG[12]	ADC	In	5V_FB	Feedback LDO 5V to check Power Supply	0V - 5V
26	PD[11]	ADC	In	MulSens_3	Current input of Motor 3	0V/0A - 5V/22.5A, Max Software: 20V
27	VDD_LV_ADC_S/ VDD_LV	POWER	In	VDD_LV_ADC_S/VDD_LV	Power Supply	5V
28	VDD_HV_IO_MAIN_2	POWER	In	VDD_HV_IO_MAIN_2	Power Supply	5V
29	PB[11]	GPIO	Out	INB_2	Signal used to detect the rotation direction of the motor 2	Logic 0 (0V) Logic 1 (5V)
30	PB[10]	ADC	In	MulSens_2	Current input of Motor 2	0V/0A - 5V/22.5A, Max Software: 20V
31	PB[9]	GPIO	Out	INA_2	Signal used to detect the rotation direction of the motor 2	Logic 0 (0V) Logic 1 (5V)
32	PB[8]	ADC	In	MulSens_1	Current input of Motor 1	0V/0A - 5V/25A, Max Software: 20V
33	VDD_LV_1	POWER	In	VDD_LV_1	Power Supply	5V
34	VDD_HV_IO_MAIN_3	POWER	In	VDD_HV_IO_MAIN_3	Power Supply	5V
35	VSS_HV_OSC	POWER	In	VSS_HV_OSC	Ground of power supply	0V
36	EXTAL	XTAL	In	EXTAL	OSC32_IN	None
37	XTAL	XTAL	In	XTAL	OSC32_OUT	None
38	VDD_HV_OSC	POWER	In	VDD_HV_OSC	Power Supply	5V
39	PA[8]	JTAG	In	TDI	Debug	None
40	PA[7]	JTAG	In	TMS	Debug	None
41	PA[9]	JTAG	In	TDO	Debug	None
42	PA[5]	JTAG	In	JCOMP	Debug	None
43	PA[6]	JTAG	In	TCK	Debug	None
44	TESTMODE	None	In/Out	TESTMODE	None	None
45	PORST	Reset	In	PORST	Positive Reset	None

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
46	PM[14]	GPIO	Out	INA_1	Signal used to detect the rotation direction of the motor 1	Logic 0 (0V) Logic 1 (5V)
47	VDD_LV_2	POWER	In	VDD_LV_2	Power Supply	5V
48	PD[5]	GPIO	Out	INB_1	Signal used to detect the rotation direction of the motor 1	Logic 0 (0V) Logic 1 (5V)
49	VDD_HV_IO_MAIN_4	POWER	In	VDD_HV_IO_MAIN_4	Power Supply	5V
50	PA[2]	Communication	Out	CAN_TX	CAN Communication	None
51	PA[1]	Communication	In	CAN_RX	CAN Communication	None
52	PA[10]	GPIO	Out	MulEN_2	Enable/ Disable MultiSense Monitoring Motor 2	Logic 0 (0V) Logic 1 (5V)
53	PA[11]	GPIO	Out	SEL0_2	Configure MultiSense Monioring for Motor 2	Logic 0 (0V) Logic 1 (5V)
54	PE[10]	GPIO	Out	PWM_3	PWM of Motor 2	Logic 0 (0V) Logic 1 (5V)
55	PE[11]	GPIO	Out	SEL1_3	Configure MultiSense Monioring for Motor 3	Logic 0 (0V) Logic 1 (5V)
56	PH[4]	GPIO	Out	SEL0_1	Configure MultiSense Monioring for Motor 1	Logic 0 (0V) Logic 1 (5V)
57	VDD_HV_FLA/VDD_HV_IO_MAIN	POWER	In	VDD_HV_FLA/VDD_HV_IO_MAIN	Power Supply	5V
58	PD[0]	GPIO	In	TW	Advanced Thermal warning output	Logic 0 (0V): Low Logic 1 (5V): High
59	PC[15]	GPIO	Out	SEL1_2	Configure MultiSense Monioring for Motor 2	Logic 0 (0V) Logic 1 (5V)
60	PC[14]	GPIO	Out	SEL1_1	Configure MultiSense Monioring for Motor 1	Logic 0 (0V) Logic 1 (5V)
61	PC[13]	GPIO	Out	PWM_2	PWM of Motor 2	Logic 0 (0V) Logic 1 (5V)

Pin No	Pin Configuration	Pin Function	Pin Direction	Signal / function	Purpose	Status Pin / Scale of Analog
62	PC[12]	GPIO	In	HALL_3	Hall Signal Motor 3	Logic 0 (0V): Low Logic 1 (5V): High
63	PF[3]	GPIO	Out	MulEN_1	Enable/ Disable MultiSense Monitoring Motor 1	Logic 0 (0V) Logic 1 (5V)
64	PF[2]			WI		
65	EP	POWER	In/Out	EP	GND	0V

3. System Architecture

3.1 System Diagram



4. Protection

5. Detail State Machine

5.1 IDLE State

Description:

System is waiting for user input or events. All motors are stopped, seat axis is updated, and the system monitors for button actions or faults to transition to other states.

Flowchart:

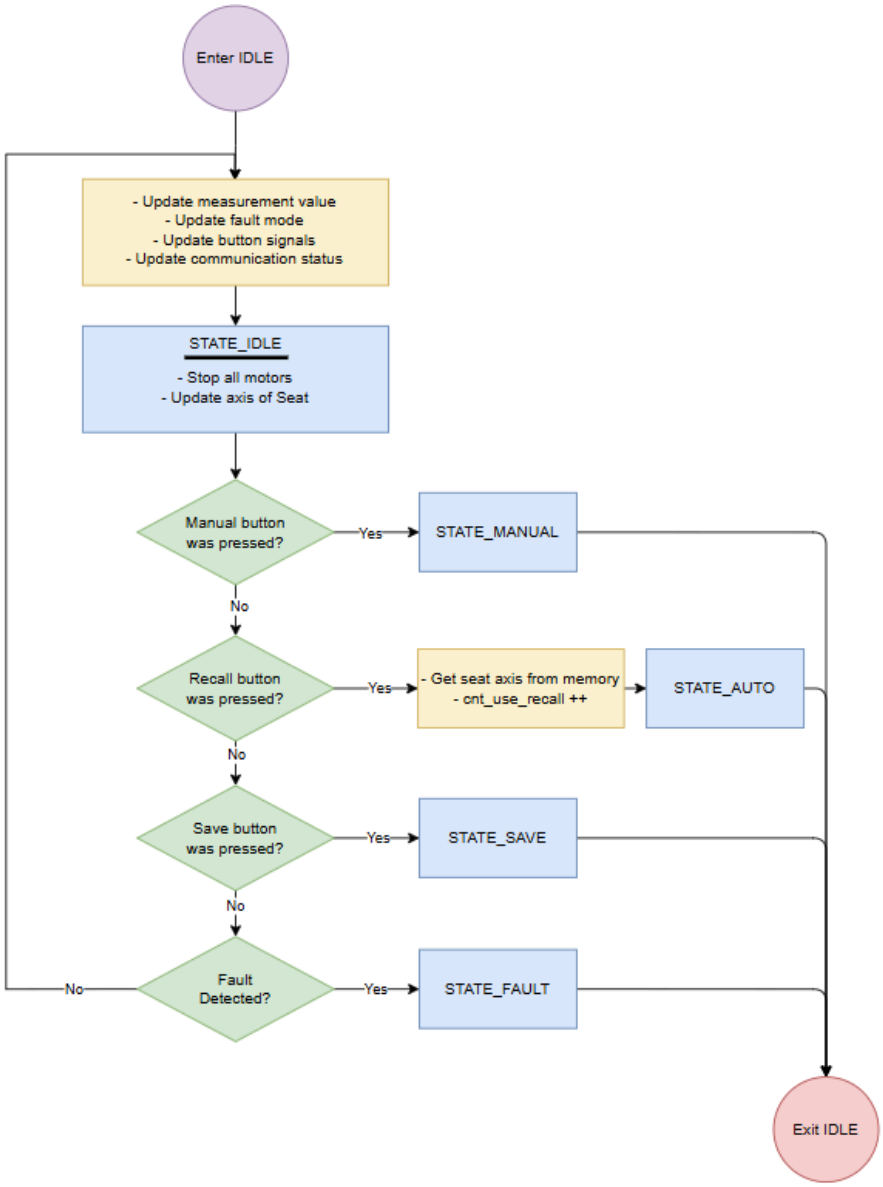


Figure 5.1 IDLE State Flowchart

Low-Level Sequence Diagram for IDLE State

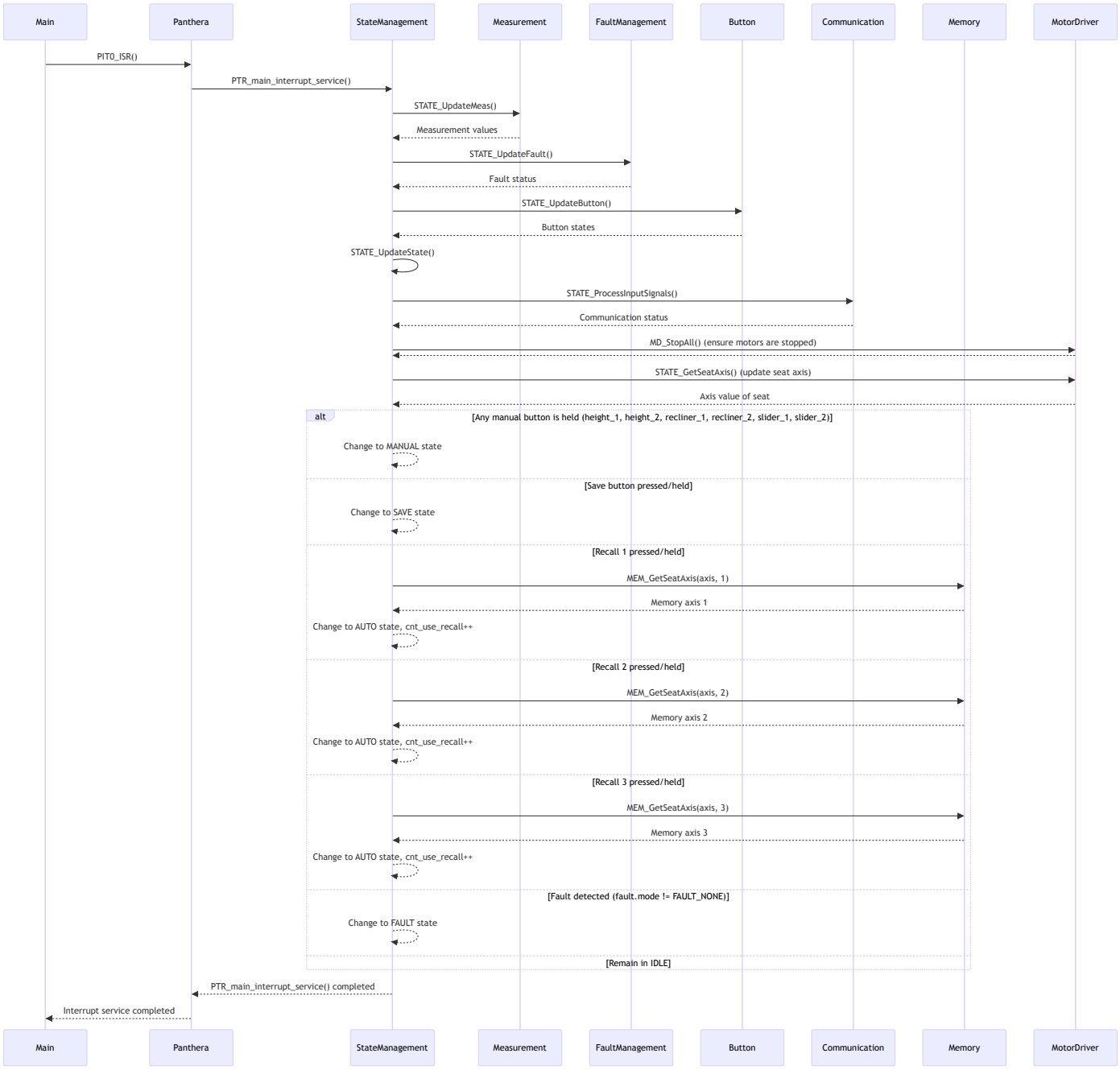


Figure 5.2 IDLE State Sequence Diagram

5.2 MANUAL State

Description:

User is manually controlling seat movement (height, recliner, slider) by holding the corresponding button. Motors run according to button actions until all are released, then return to IDLE.

Flowchart:

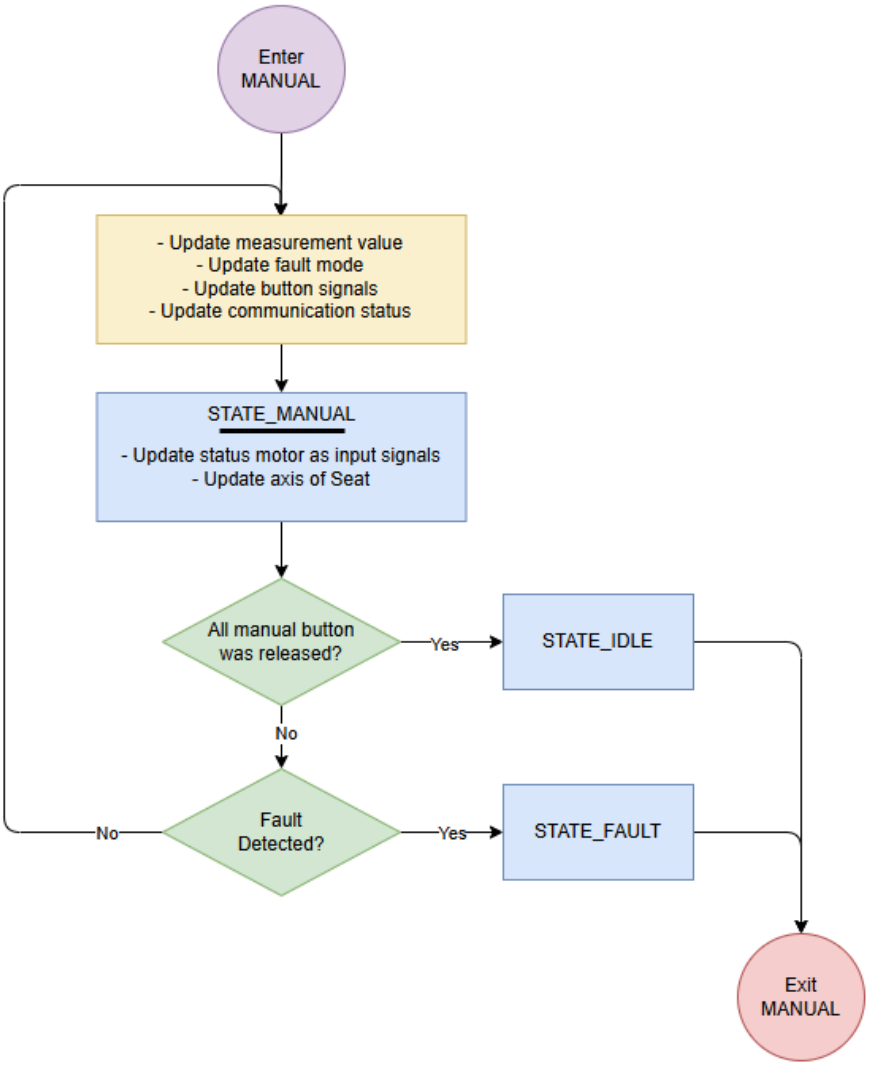


Figure 5.3 MANUAL State Flowchart

Low-Level Sequence Diagram for MANUAL State

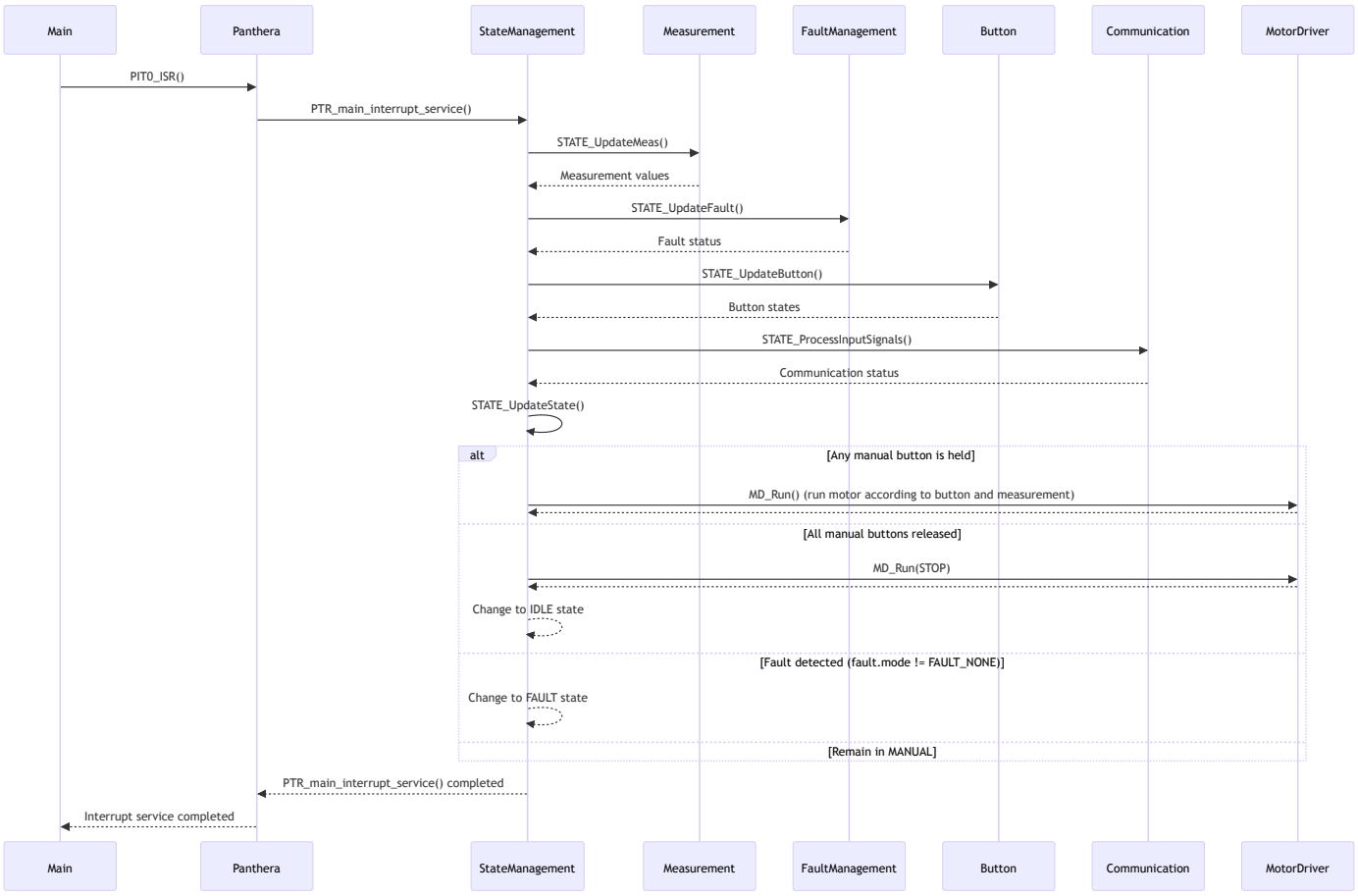


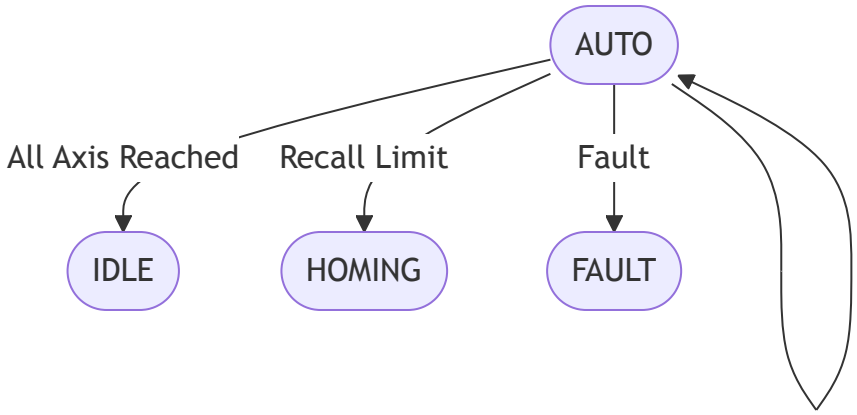
Figure 5.4 MANUAL State Sequence Diagram

5.3 AUTO State

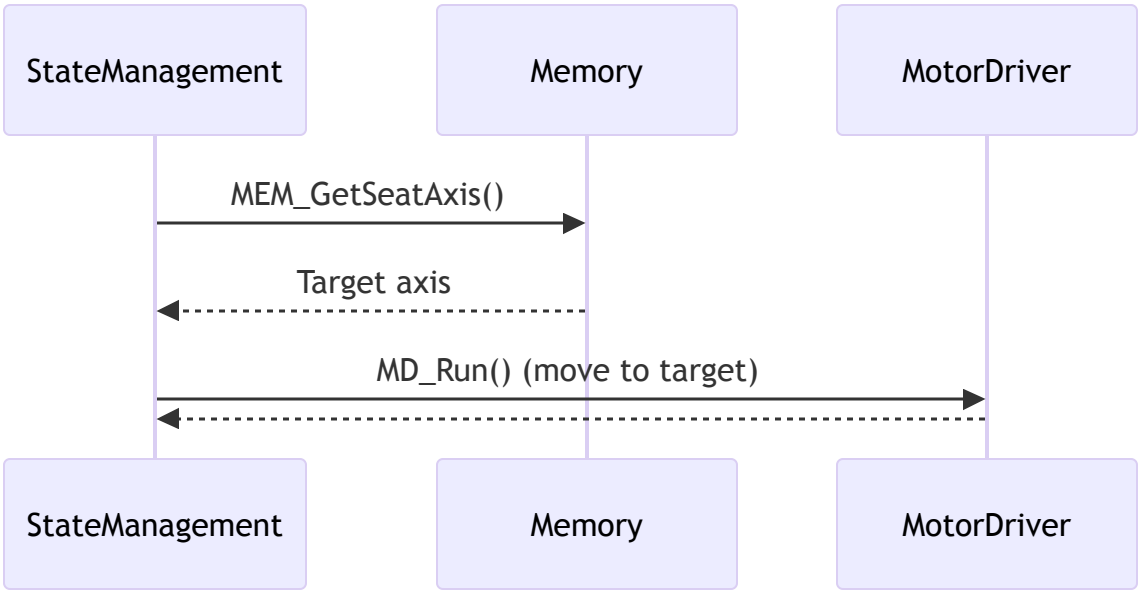
Description:

System automatically moves seat to a saved position (recall 1/2/3). Motors are controlled to reach target axis values from memory. When all axes reach their targets, returns to IDLE or transitions to HOMING if recall count limit is reached.

Flowchart:



Sequence Diagram:

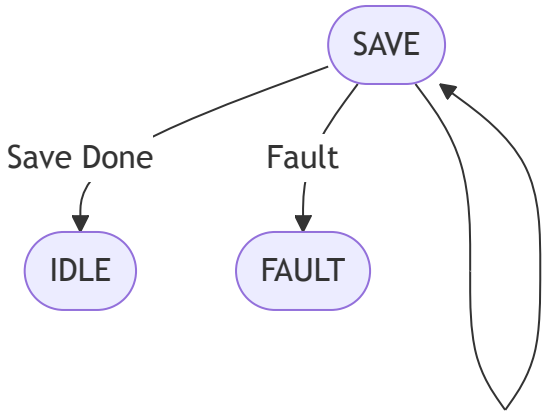


5.4 SAVE State

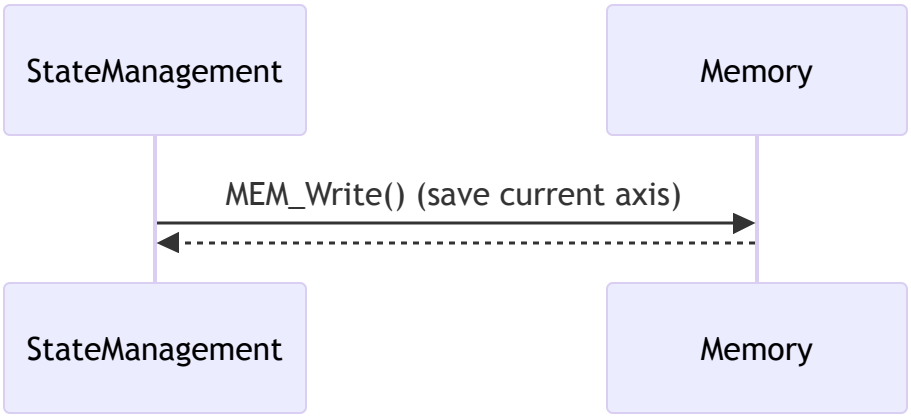
Description:

User requests to save current seat position. System writes current axis values to memory. After save and a delay, returns to IDLE.

Flowchart:



Sequence Diagram:

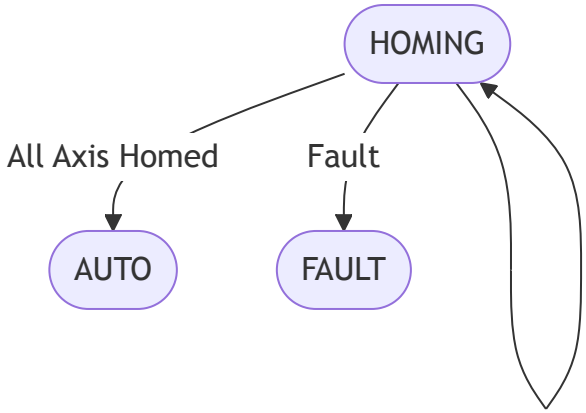


5.5 HOMING State

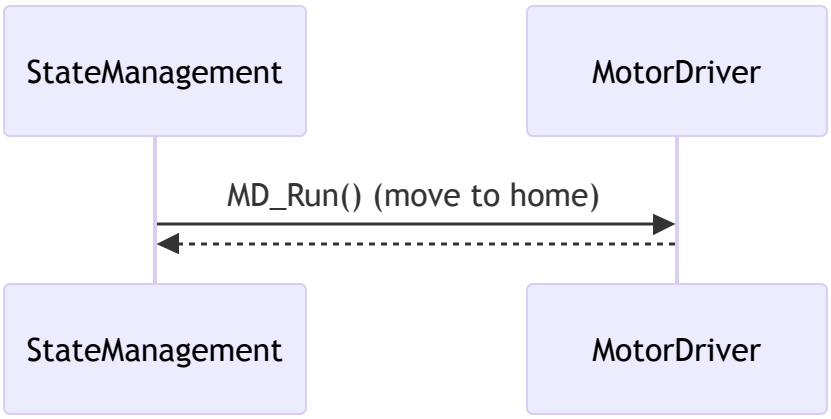
Description:

System moves seat axes to their home (reference) positions for calibration or after multiple recalls. When all axes reach home, returns to AUTO and resets axis tracking.

Flowchart:



Sequence Diagram:

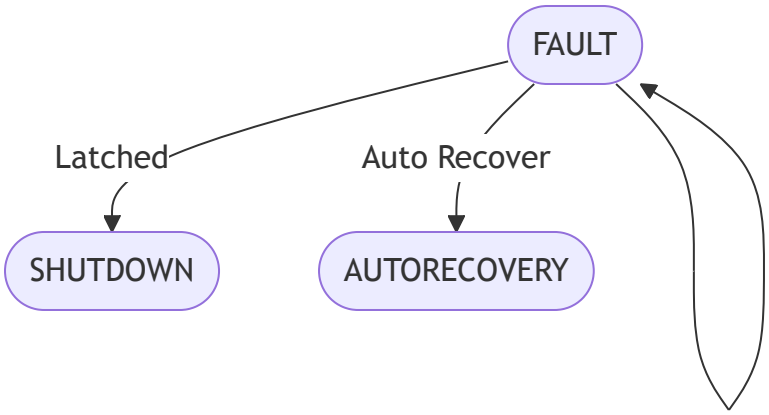


5.6 FAULT State

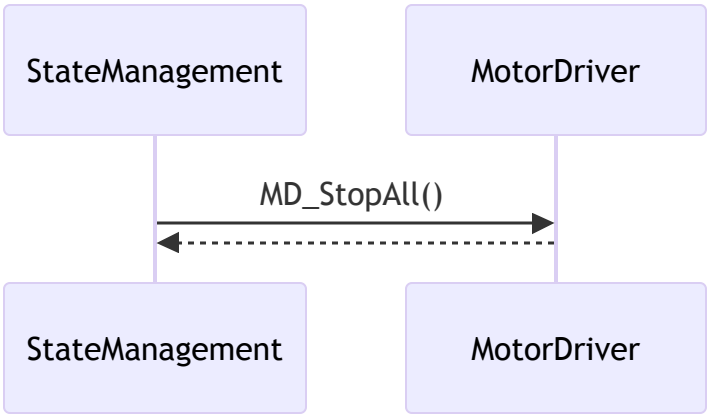
Description:

A fault is detected (overcurrent, thermal, etc.). All motors are stopped for safety. System waits for fault to clear or transitions to SHUTDOWN/AUTORECOVERY depending on fault type.

Flowchart:



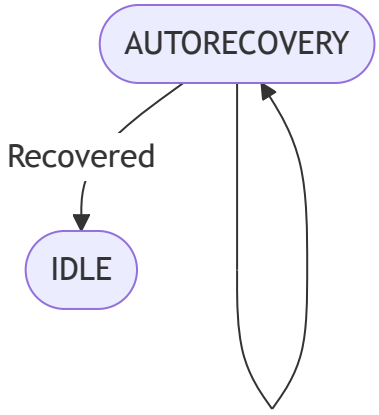
Sequence Diagram:



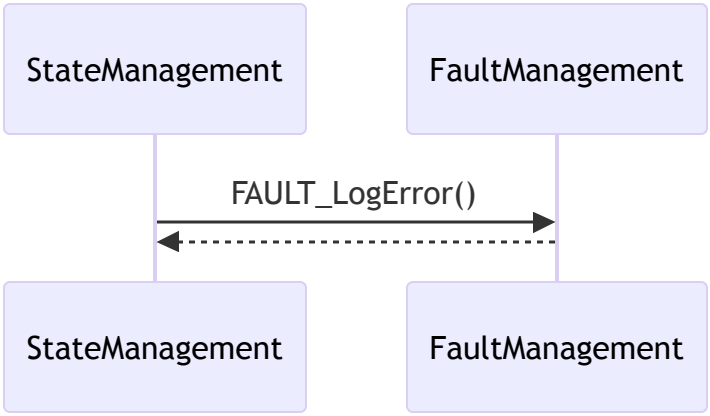
5.7 AUTORECOVERY State

Description:
System attempts to recover from a recoverable fault. If system returns to normal, transitions back to IDLE and resets fault mode.

Flowchart:



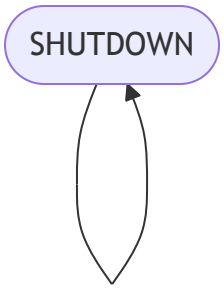
Sequence Diagram:



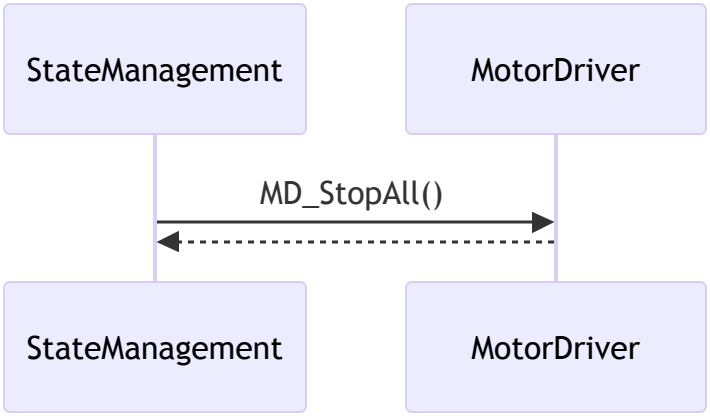
5.8 SHUTDOWN State

Description:
System enters safe shutdown due to a critical or latched fault. Motors remain stopped and system awaits reset or power cycle.

Flowchart:



Sequence Diagram:



6. Api Summary

6.1 Driver

No.	API Function Signature	Layer	Library	Description
1	int ADC_Init(ADC_TypeDef *pADC)	Driver	adc	Initialize ADC structure and driver
2	void ADC_startOfConversion(void)	Driver	adc	Start ADC conversion for configured channels
3	void saradcconf_conv_cb(SARADCDriver *saradcp)	Driver	adc	Callback for ADC conversion complete
4	void saradcconfdma_conv_cb(SARADCDriver*, uint16_t*, uint16_t)	Driver	adc	Callback for DMA-based ADC conversion (placeholder)
5	void EEPROM_Start(void)	Driver	eeprom_user	Initialize EEPROM
6	void EEPROM_Restart(void)	Driver	eeprom_user	Erase all data and restart EEPROM
7	void EEPROM_Write(uint32_t adc_value, uint16_t id)	Driver	eeprom_user	Write 32-bit value to EEPROM with specified ID
8	uint32_t EEPROM_Read(uint16_t id)	Driver	eeprom_user	Read 32-bit value from EEPROM by ID
9	void GPIO_SetPin(GPIO_TypeDef *pPin)	Driver	gpio	Set GPIO pin to high
10	void GPIO_ClearPin(GPIO_TypeDef *pPin)	Driver	gpio	Set GPIO pin to low
11	void GPIO_TogglePin(GPIO_TypeDef *pPin)	Driver	gpio	Toggle GPIO pin state

No.	API Function Signature	Layer	Library	Description
12	uint8_t GPIO_ReadPin(GPIO_TypeDef *pPin)	Driver	gpio	Read GPIO pin state (0/1)
13	void GPIO_Init(void)	Driver	gpio	Initialize all GPIO pins used in the system
14	void GPIO_Update(void)	Driver	gpio	Update all GPIO pin values to global structure
15	void UART_Init(void)	Driver	uart	Initialize UART, buffer, and state
16	void UART_HandleRequest(void)	Driver	uart	Handle UART data transmission/reception and status
17	void UART_TX_Callback(SerialDriver *sdp)	Driver	uart	Callback when UART transmission is complete
18	void *sbrk(ptrdiff_t incr)	Driver	uart	Heap memory allocation function (for malloc, weak symbol)

6.2 Middleware

No.	API Function Signature	Layer	Library	Description
1	void BTN_GetGpioValue(GPIO_Input_TypeDef *pGpio)	Middleware	Button	Update button values from GPIO inputs
2	void BTN_Init(void)	Middleware	Button	Initialize all buttons in the system
3	void BTN_InitBtn(Btn_Object_TypeDef *pButton, Btn_Name_TypeDef name)	Middleware	Button	Initialize a single button object
4	void BTN_Analyze(Btn_Object_TypeDef *pButton)	Middleware	Button	Analyze button state (debounce, pressed, hold)
5	void BTN_UpdateToState(Button_TypeDef *pButtonTemp, Btn_Object_TypeDef *pButton)	Middleware	Button	Update button state to system structure
6	void BTN_Update(ButtonSystem_TypeDef *pButton)	Middleware	Button	Update all button states
7	void COMM_Init(void)	Middleware	communication	Initialize communication module
8	void COMM_CommToState(CommSystem_TypeDef *pComm)	Middleware	communication	Update communication state
9	void COMM_TransmitReceiveHandle(void)	Middleware	communication	Handle transmit/receive process
10	void COMM_Handle(void)	Middleware	communication	Main communication handler

No.	API Function Signature	Layer	Library	Description
11	void COMM_DecodeMessage(void)	Middleware	communication	Decode received message
12	void COMM_UpdateStatus(void)	Middleware	communication	Update status from system to communication
13	void COMM_Respone(void)	Middleware	communication	Send response message
14	void COMM_HandleWriteRequest(void)	Middleware	communication	Handle write request from communication
15	void FAULT_Init(void)	Middleware	FaultManagement	Initialize fault management module
16	void FAULT_Handle(FaultSystem_TypeDef *pFault, MeasSystem_TypeDef *pMeas)	Middleware	FaultManagement	Handle fault management process
17	void FAULT_LogError(void)	Middleware	FaultManagement	Log fault error
18	void FAULT_Reset(void)	Middleware	FaultManagement	Reset fault status
19	void FAULT_UpdateStatus(MeasSystem_TypeDef *pMeas)	Middleware	FaultManagement	Update fault status based on measurements
20	void FAULT_UpdateMode(Fault_TypeDef *pFault, MeasSystem_TypeDef *pMeas)	Middleware	FaultManagement	Update fault mode based on status
21	void MEAS_Config(Meas_TypeDef *pMeas, q15_t offset, uq31_t scale_factor, uq15_t alpha, uq15_t high_limit_soft, float gain_f32, float offset_f32, uint8_t calc_method)	Middleware	Measurement	Configure measurement object
22	void MEAS_GetRawValue(Meas_TypeD	Middleware	Measurement	Set raw value in measurement

No.	API Function Signature	Layer	Library	Description
	ef *pMeas, q15_t raw_value)			structure
23	void MEAS_GetValueQ15(Meas_TypeDef *pMeas)	Middleware	Measurement	Calculate Q15 value for measurement
24	void MEAS_GetValueF32(Meas_TypeDef *pMeas)	Middleware	Measurement	Calculate float value for measurement
25	void MEAS_GetValue(MeasType_TypeDef meas_type)	Middleware	Measurement	Get measurement value by type
26	void MEAS_GetRealValue(MeasType_TypeDef meas_type)	Middleware	Measurement	Get real (physical) value by type
27	uint16_t MEAS_LPF(uint16_t new_value, uint16_t prev_value, uint16_t alpha)	Middleware	Measurement	Low-pass filter for measurement
28	void MEM_Read(uint16_t id_recall)	Middleware	Memory	Read memory by recall ID
29	void MEM_Write(uint32_t x, uint32_t y, uint32_t z, uint16_t id_recall)	Middleware	Memory	Write memory with coordinates and recall ID
30	void MEM_GetSeatAxis(SeatAxis_TypeDef *pSeat, uint16_t id_recall)	Middleware	Memory	Get seat axis from memory by recall ID
31	void MD_Init(void)	Middleware	MotorDriver	Initialize motor driver module
32	void MD_StopAll(void)	Middleware	MotorDriver	Stop all motors
33	void MD_Run(MotorName_TypeDef motor_name, MotorStatus_TypeDef status, uint16_t current_value)	Middleware	MotorDriver	Run motor with specified status and current
34	void MD_ConfigDriver(Motor_TypeDef	Middleware	MotorDriver	Configure motor driver pins

No.	API Function Signature	Layer	Library	Description
	*pMotor, GPIO_TypeDef pin_ina, GPIO_TypeDef pin_inb, GPIO_TypeDef pin_sel0, GPIO_TypeDef pin_sel1, GPIO_TypeDef pin_mul_en, GPIO_TypeDef pin_pwm)			
35	void MD_ConfigTopology(Motor_TypeDef *pMotor, MotorTopology_TypeDef topology)	Middleware	MotorDriver	Configure motor topology
36	void MD_UpdateDriver(Motor_TypeDef *pMotor)	Middleware	MotorDriver	Update motor driver pins
37	void MD_UpdateStatusMotor(Motor_TypeDef *pMotor, MotorStatus_TypeDef status)	Middleware	MotorDriver	Update motor status
38	void MD_UpdateMeasType(Motor_TypeDef *pMotor, MeasType_TypeDef meas_type)	Middleware	MotorDriver	Update measurement type for motor
39	_Bool MD_DetectHallSignalRising(Motor_TypeDef *pMotor)	Middleware	MotorDriver	Detect rising edge of hall signal
40	_Bool MD_DetectPeak(Motor_TypeDef *pMotor)	Middleware	MotorDriver	Detect peak in sensorless mode
41	_Bool MD_ScaleResolutionSensor(Motor_TypeDef *pMotor)	Middleware	MotorDriver	Scale resolution for sensor mode
42	_Bool MD_ScaleResolutionSensorless(Motor_TypeDef *pMotor)	Middleware	MotorDriver	Scale resolution for sensorless mode

No.	API Function Signature	Layer	Library	Description
43	uint16_t MD_RoundAxis(Motor_TypeDef *pMotor)	Middleware	MotorDriver	Round axis value for motor

6.3 Application

No.	API Function Signature	Layer	Library	Description
1	void Panthera_Init(void)	Application	Panthera.c	Initialize the entire Panthera system
2	void Panthera_Run(void)	Application	Panthera.c	Main application loop
3	void Panthera_SetSeatPos(void)	Application	Panthera.c	Set seat position as requested by user
4	void Panthera_SaveSeatPos(void)	Application	Panthera.c	Save current seat position
5	void Panthera_LoadSeatPos(void)	Application	Panthera.c	Load saved seat position
6	void Panthera_HandleError(void)	Application	Panthera.c	Handle errors during operation
7	void STATE_Init(PTR_TypeDef *pState)	Application	StateManagement	Initialize state management module
8	void STATE_UpdateState(PTR_TypeDef *pState)	Application	StateManagement	Update system state
9	void STATE_UpdateMeas(PTR_TypeDef *pState)	Application	StateManagement	Update measurement in state management
10	void STATE_UpdateFault(PTR_TypeDef *pState)	Application	StateManagement	Update fault in state management
11	void STATE_UpdateButton(PTR_TypeDef *pState)	Application	StateManagement	Update button in state management
12	void STATE_idleHandle(PTR_TypeDef *pState)	Application	StateManagement	Handle IDLE state

No.	API Function Signature	Layer	Library	Description
13	void STATE_saveHandle(PTR_TypeDef *pState)	Application	StateManagement	Handle SAVE state
14	void STATE_manualHandle(PTR_TypeDef *pState)	Application	StateManagement	Handle MANUAL state
15	void STATE_autoHandle(PTR_TypeDef *pState)	Application	StateManagement	Handle AUTO state
16	void STATE_faultHandle(void)	Application	StateManagement	Handle FAULT state
17	void STATE_autorecoveryHandle(void)	Application	StateManagement	Handle AUTORECOVERY state
18	void STATE_shutdownHandle(void)	Application	StateManagement	Handle SHUTDOWN state
19	void STATE_homingHandle(PTR_TypeDef *pState)	Application	StateManagement	Handle HOMING state
20	void STATE_ProcessInputSignals(PTR_TypeDef *pState)	Application	StateManagement	Process input signals
21	_Bool STATE_SystemIsNormal(PTR_TypeDef *pState)	Application	StateManagement	Check if system is normal
22	void STATE_GetSeatAxis(PTR_TypeDef *pState)	Application	StateManagement	Get seat axis position
23	_Bool STATE_CheckEndPosition(uint32_t *pCurrent, uint32_t *pPrev)	Application	StateManagement	Check end position for seat axis
24	void STATE_ResetAxis(PTR_TypeDef	Application	StateManagement	Reset seat axis

No.	API Function Signature	Layer	Library	Description
	*pState)			

Revision Note

Version	Date	Author	Description
0.0	2025-08-18	Quoc Uy Nguyen	Initial documentation